

Section 1. Data Set & Problem Definition

Problem Definition

The primary objective of this project is to develop and evaluate a multi-class classification system for autonomous driving perception. Specifically, the task involves identifying 43 distinct categories of road signs from visual data. Traffic sign recognition is a critical component of autonomous vehicles, as these systems must interpret traffic signs to navigate safely and legally.

This problem is particularly interesting because it mirrors the high-stakes environment of real-world driving. A model must handle significant visual variance, including changes in lighting, motion blur, and partial occlusions. I chose this problem due to the safety-critical nature of the task; a failure to distinguish a "Stop" sign from a "Yield" sign can result in fatal accidents.

Dataset Summary

The dataset utilized for this report is the **German Traffic Sign Recognition Benchmark (GTSRB)**, specifically the version hosted on Hugging Face by [dpdl-benchmark/gtsrb](https://huggingface.co/datasets/dpdl-benchmark/gtsrb). This dataset contains ~40,000 images of traffic signs belonging to 43 classes. The data was loaded into a structured format, resulting in a training set of 21,312, a validation set of 5,328 and a test set of 12,630 images.

The class distribution reveals a significant imbalance; for instance, common signs like "Speed Limit 50km/h" have nearly 1,200 samples, whereas rarer signs like "Attention Left Turn" have 120 samples. This lack of balance is a realistic reflection of road conditions, where certain signs are encountered more frequently than others. A visual inspection of the samples highlights several practical limitations. First, many images are provided at a low resolution, which removes fine-grained details such as subtle weathering or text on supplementary signs. Second, there are significant sources of confusion within the dataset; categories such as different speed limits (e.g., 30km/h vs. 50km/h) or similar triangular warning signs are visually highly correlated, especially under poor lighting or blur.

Random GTSRB samples



Section 2. Embedding Model + Transfer Learning

Overview of Approach

For the GTSRB task, I implemented a transfer learning pipeline utilizing a high-performance pretrained vision backbone. The core strategy relies on using a foundation model as a fixed feature extractor, which allows the system to benefit from large-scale pretraining on diverse visual data without the computational cost of full-parameter fine-tuning.

Frozen Backbone + Classification Head

The chosen encoder is **DINOv2** ([facebook/dinov2-large](#)), a self-supervised vision transformer model known for its highly discriminative local and global features. The model was kept in a frozen state, acting as a static encoder. I extracted the **[CLS]** token embedding for each traffic sign image, compressing the visual information into a single numerical vector that represents the image's global semantic content.

On top of these frozen embeddings, I built a **custom MLP** using PyTorch. This classification head was trained to map the static feature vectors to the 43 target traffic sign categories.

Architecture and Training

- **Pretrained Encoder:** I utilized [facebook/dinov2-large](#).
- **Embedding Dimensionality:** The DINOv2-large backbone produces embeddings with a dimensionality of **1024**. These were extracted from the [last_hidden_state](#) of the transformer.
- **Network Architecture:** The PyTorch classifier is a three-layer MLP:
 - **Input Layer:** 1024 units (matching the DINOv2 embedding size).
 - **Hidden Layer 1:** 512 units with ReLU activation and a Dropout rate of 0.2.
 - **Hidden Layer 2:** 128 units with ReLU activation and a Dropout rate of 0.2.
 - **Output Layer:** 43 units (Softmax cross-entropy loss) to correspond with the GTSRB classes.
- **Training Process:** The model was trained for **30 epochs** using the **Adam optimizer** with a learning rate of **0.001** and a batch size of **256**. I implemented **early stopping** based on validation loss to prevent overfitting. This approach ensures that the model halts training once it reaches its peak generalization capability on the validation set.

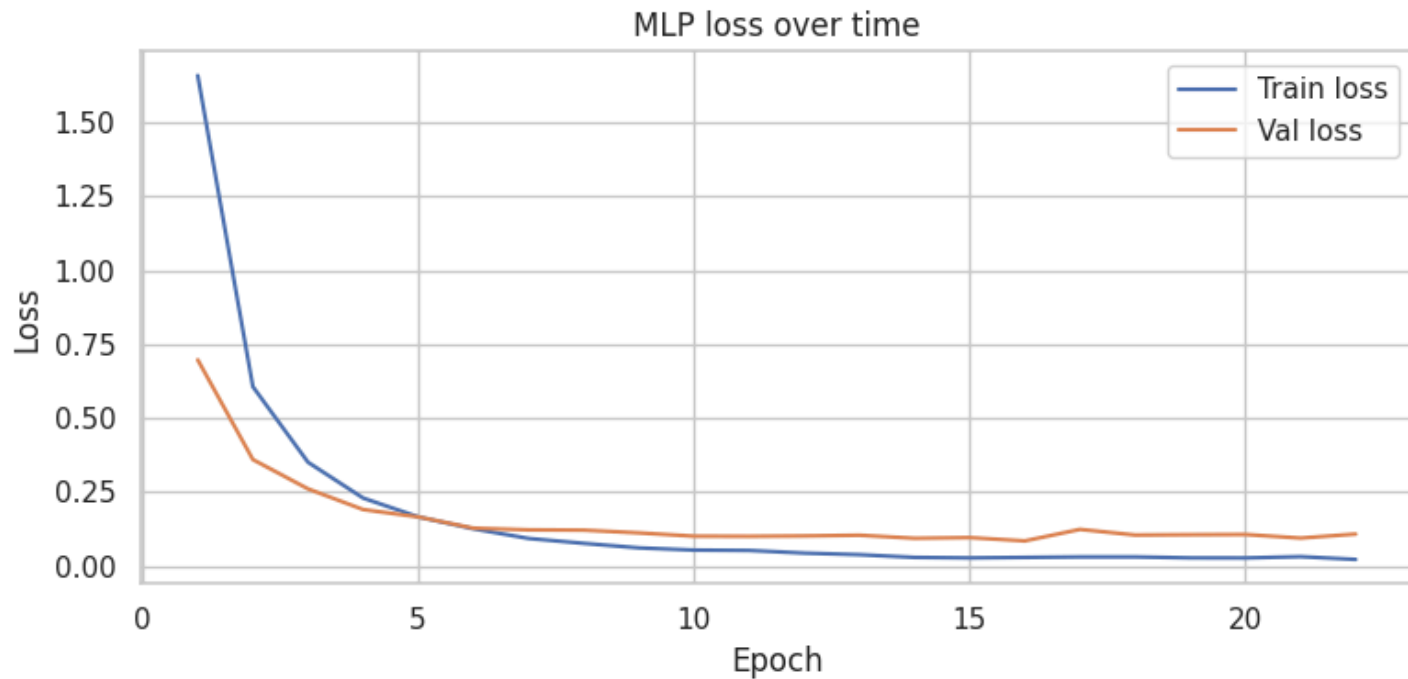
Training and Validation Loss

An analysis of the training reveals that the validation loss closely followed the training loss for the majority of the training duration, indicating a healthy learning process without immediate overfitting. The model effectively reached a point of convergence around epoch 22, where the rate of improvement in validation loss began to diminish.

Maximum epoch count was set to 30, but the early stopping mechanism stopped the training. This prevented the model from beginning to over-fit the training distribution in the final few iterations, and stored the weights that represent the most generalizable state of the network. The synchronization between the two loss curves suggests that the 1024-dimensional DINOv2 embeddings are

3

exceptionally robust, providing a linearly separable space that the MLP could navigate with minimal complexity.



Section 3. Large-Language Model

LLM Interface

To establish a baseline using a foundation model with general intelligence, I utilized **Qwen3-VL-2B-Instruct**. This is a Vision-Language Model (VLM) designed to process both image and text inputs natively.

I chose the **Hugging Face Transformers** implementation to run the model directly in the development environment. This allowed for integration with the existing data pipeline while maintaining high precision using **bfloat16** on the GPU. The 2B-parameter version was selected specifically for its balance between visual reasoning and inference efficiency.

LLM Prompting

I employed a **zero-shot label-constrained prompting** strategy. The model was asked to classify the traffic sign image without being shown any prior examples in the prompt, relying entirely on its pre-trained internal knowledge.

Exact Prompt Submitted:

*"You are classifying a GTSRB image. Choose exactly one label from: [List of 43 Classes].
Return only the label word."*

To ensure the model adhered to the task, I gave the list of 43 classes to match the official GTSRB taxonomy. I chose zero-shot prompting because the goal was to evaluate the out-of-the-box general intelligence of the foundation model.

Analysis

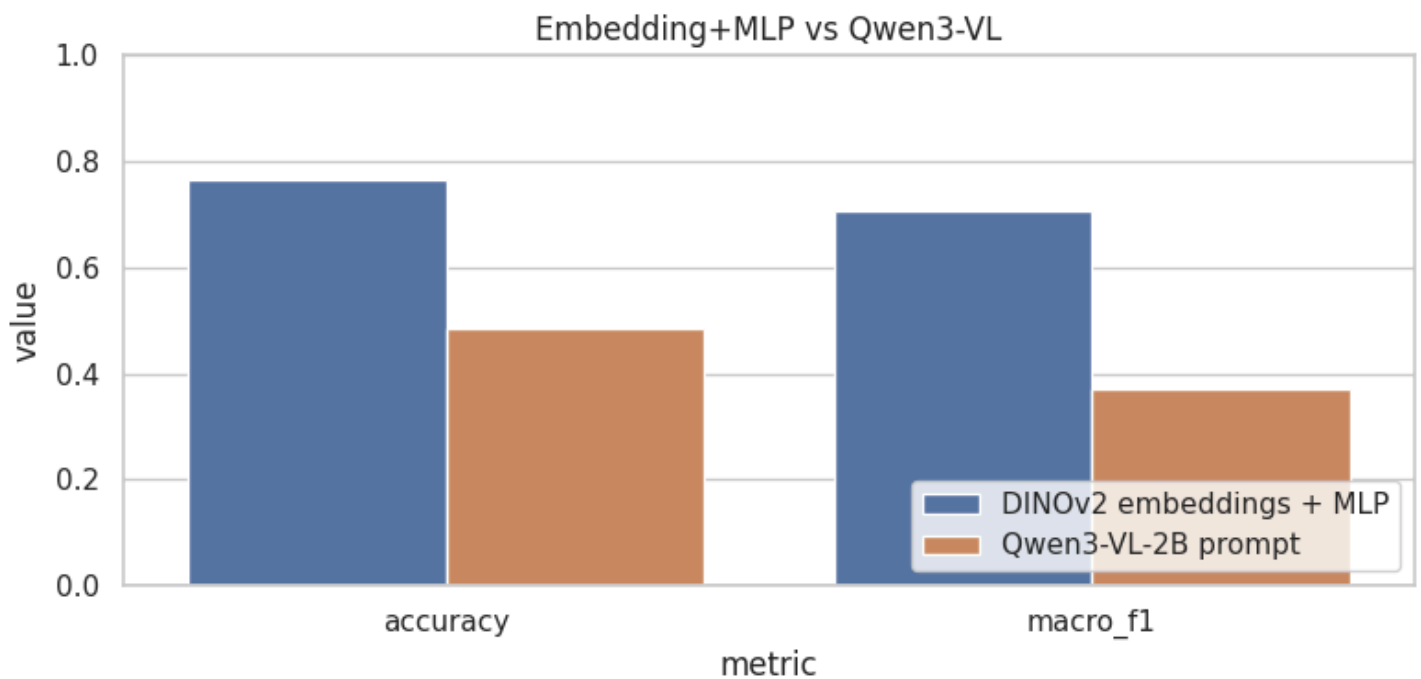
- **Label Extraction:** I utilized a Regular Expression matching system to extract the final class label from the model's response. This system scanned the model's output for any of the 43 valid class strings, allowing for successful parsing even when the model included conversational filler.
- **Evaluation Subset:** Due to the computational intensity of VLM inference compared to the MLP classifier, I evaluated the LLM on a random subsample of **200 examples** from the testing set. This provided a statistically significant baseline for comparison while keeping inference manageable.
- **Tuning Observations:** Initial attempts without class constraints led the model to provide overly descriptive answers (e.g., "A red circular sign with a white horizontal bar" instead of "No entry"). Providing the exact target vocabulary was the most effective tuning step to bridge the gap between human language and structured classification.

Section 4. Evaluation

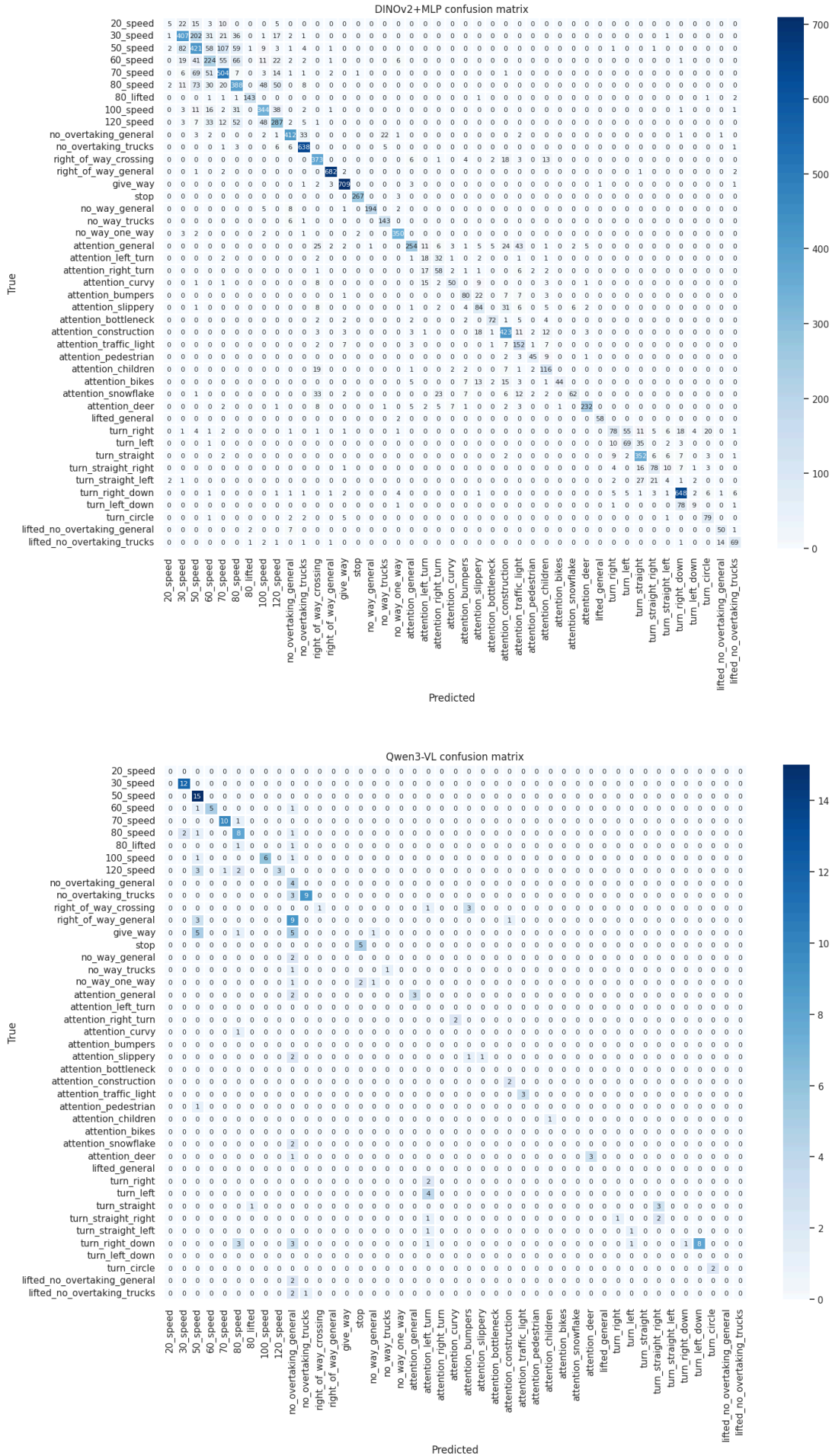
Comparative Evaluation

The empirical results reveal a substantial performance gap between the two approaches. The **DINOv2 + MLP pipeline achieved an accuracy of ~76.6%** on the full test set, whereas the **Qwen3-VL-2B zero-shot baseline achieved only ~48.5%** on the 200-sample evaluation subset.

This ~28% disparity highlights the immense value of specialized, task-specific training. The VLM relied entirely on its generalized pre-training and zero-shot reasoning. Current literature shows that VLMs often struggle with strict categorical mappings and fine-grained visual distinctions in low-resolution images, which explains the lower zero-shot accuracy.



Class-Specific Behavior: As observed in the confusion matrices, the models exhibited divergent behaviors. The embedding model showed strong clustering along the diagonal but suffered from localized confusion among visually similar classes (e.g., various speed limit signs). The VLM displayed a broader, more scattered confusion pattern, struggling particularly with arrows that look confusing.



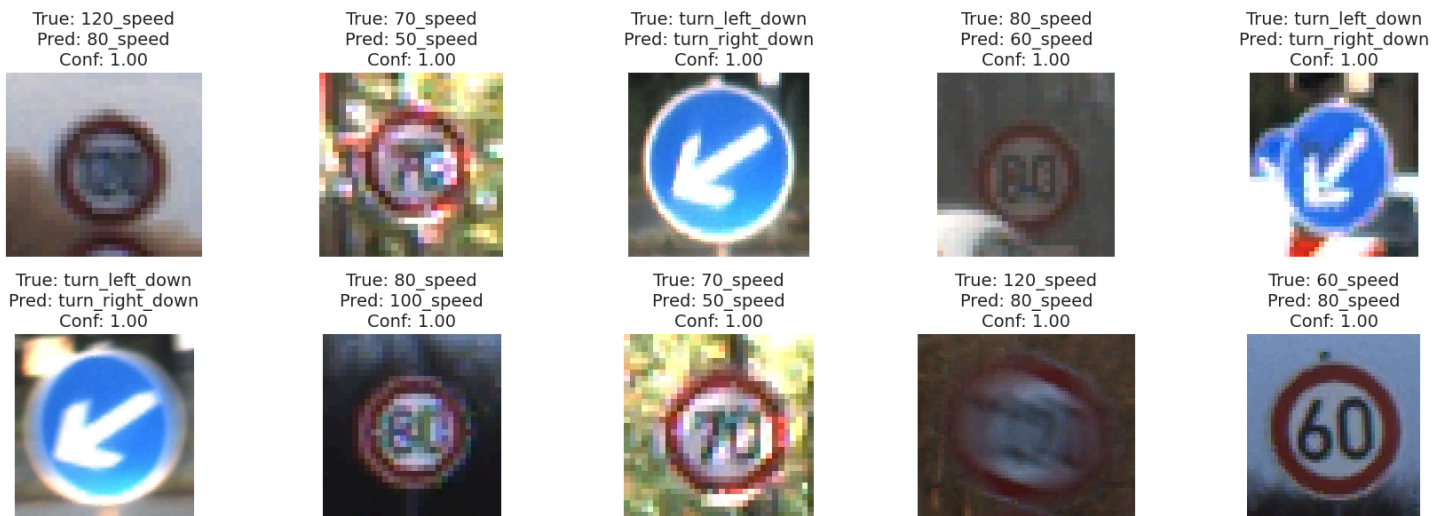
Failure Case Analysis

Embedding Model (DINOv2 + MLP)

Confusion between similar regulatory signs, such as "Speed limit (80km/h)" versus "Speed limit (60km/h)". This is primarily a resolution limitation. The macroscopic features (red circular border, white interior) dominate the DINOv2 embedding. If motion blur or lighting obscures the central '6' or '8', the global embedding vector cannot distinctly separate the classes.

One solution is to implement targeted data augmentation (e.g., random cropping, contrast enhancement) specifically focused on the center of the signs to force the network to prioritize the fine-grained alphanumeric features over the global shape.

MLP common misclassifications



VLM Baseline (Qwen3-VL)

Semantic and directional mistakes, such as confusing "Turn left ahead" with "Turn right ahead". VLMs are known to exhibit weaknesses in spatial orientation and frame-of-reference reasoning. Without specific contextual grounding, the model may misinterpret the geometric direction of the arrow. Furthermore, zero-shot prompting is highly sensitive.

One solution is to transition from zero-shot to **few-shot prompting** (providing 2-3 examples of left vs. right signs in the prompt) would anchor the model's spatial reasoning.

Qwen3-VL misclassifications



Engineering Trade-offs

When designing a real-world autonomous perception stack, the choice between these architectures involves balancing several competing constraints:

1. **Efficiency and Latency:** The embedding model is drastically more efficient. Aside from the one-time cost of passing the image through the DINOv2 backbone, the MLP inference takes fractions of a millisecond. In contrast, the VLM incurs high latency and heavy GPU compute costs per inference, making it largely unsuitable for the strict real-time (~30+ FPS) constraints of an in-vehicle system.
2. **Implementation Complexity vs. Extensibility:** The MLP approach requires a rigid data collection pipeline, hyperparameter tuning, and retraining the head every time a new class is introduced. The VLM shines in its extensibility. If a vehicle crosses into a region with 10 new, unseen traffic signs, updating the VLM system merely requires modifying the text prompt with the new labels, so zero retraining required.
3. **Reliability:** The MLP provides deterministic, structured outputs. The VLM, due to its autoregressive generative nature, introduces a layer of fragility (parsing errors, hallucinations) that requires heavy engineering guardrails to deploy safely.

Conclusion: For a specialized, safety-critical task like traffic sign classification under strict hardware and latency constraints, the traditional **frozen embedding + trained classifier** paradigm remains the vastly superior engineering choice. VLMs are currently better suited for offline data annotation or complex, open-ended scene understanding where rigid classes cannot be defined prior.